XRPL Validator — Daily Engineering Report · 2026-10-23

Period: 2026-10-23 00:00 to 24:00 US Eastern (2026-10-23 04:00 to 2026-10-24 04:00 UTC); ledgers #107,841,424 to #107,863,468. Every count from the ledger and the validator's logs stops at #107,863,468, closed 03:59:52 UTC. Generated 2026-10-24 00:21 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Status

Normal — coverage outside planned maintenance 100.000% (objective 99.9%), every validation agreed with the network, no open engine receipt.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** signed 22,045 of the 22,045 ledgers the network closed (100.000%); every validation matched the network's validated ledger.
- **Native engine:** 0 engine receipts.
- **Changes:** 0 deployments, 108 commits merged into main.
- **Network:** 1 slow-close period; the longest, 16:43 UTC–16:44 UTC, closed ledgers up to 19 s apart.
- **Independence program:** 0 of 30 qualifying days (snapshot at 04:21:16 UTC).
- **Amendment fixCleanup3_4_0** holds majority; activation due ~2026-11-04 14:46 UTC.

Service levels

Measure	Value
Ledgers closed by the network	22,045
Validations issued	22,045
Coverage (validations / ledgers closed)	100.000%
Validations matching the network's validated ledger hash	22,045 of 22,045
Unsigned during planned restarts	0
Unsigned outside maintenance	0
Coverage outside planned maintenance (objective 99.9%)	100.000%
Restarts	0
Basis: our own ledger hash	22,045
External agreement, data.xrpl.org (UTC day 2026-10-23)	1.00000 (0 missed of 21,117)

The basis rows sum to validations issued. Coverage counts the validations the validator issued for ledgers the network closed in the period. Agreement compares each validation's ledger hash with the network's validated hash for that ledger. data.xrpl.org scores UTC days from the validations its crawlers received, so it can differ.

Native engine conformance

Ledgers with a receipt in the period (#107,841,424–#107,863,468), by step:

Step	Ledgers with a receipt
Step 1 · native engine	0
Step 2 · records	0
Step 3 · order	0
Step 4 · peer data	0
Step 4 · validations	0
Step 5 · own hash	0
Writer	0

Live counters, snapshot at 04:21:16 UTC (since the last restart, 2026-10-22 12:09 UTC; not cut off at the period's last ledger):

Step	Checked	Result
1 · native engine vs libxrpl	36,717 ledgers, 5,976,877 transactions	36,717 fully matched; results 0, bytes 0, keys 0/0
2 · transaction records	36,759 ledgers, 6,166,737 records	6,166,737 exact; differ 0, tree 0
3 · transaction order	36,759 ledgers (mode: on)	36,759 exact; order 0, results 0, bytes 0, refused 0
4 · ledger data from peers	36,743 ledgers fetched	36,743 matched; differ 0, failed 0
4 · trusted validations	36,831 ledgers judged	36,831 agreed; differ 0, short of quorum 0
5 · our own ledger hash	36,833 ledgers	36,759 own; differ 0, partial 74
Writer	every ledger written	mismatch 0, distrusted 0

"Partial" ledgers are those the engine could not yet build alone while warming up after a restart.

Receipt register

No receipts this day.

Incidents

No incidents.

Change log

Deployments

No deployments.

Merged into main

Merged	Repo	Commit	Subject
02:41 UTC	xrpl-sdk	`55fb80b`	fix(core): the encoder refuses what rippled refuses instead of changing the transaction
02:41 UTC	xrpl-sdk	`37dc627`	fix(core): token values parse and round exactly as rippled reads them
02:41 UTC	xrpl-sdk	`1ef2462`	fix(client): submit_and_wait answers only with final outcomes; autofill caps the fee
02:41 UTC	xrpl-sdk	`1a10b5a`	fix(crypto): secrets stay out of logs and errors; secp256k1 signs only a 32-byte hash
02:41 UTC	xrpl-sdk	`b5fd628`	fix(cli): the server decides whether --yes is needed; failed sends exit non-zero
02:41 UTC	xrpl-sdk	`cb9c67c`	fix(client): the WebSocket client closes on drop, notices dead connections, and routes each subscription its own events
02:41 UTC	xrpl-sdk	`58116da`	fix(transaction): Batch checks what rippled checks; a transaction that does not parse is Unparsed, not Other
02:41 UTC	xrpl-sdk	`eafb7ae`	fix(core): canonical currencies, minimal field headers, capped base58, checked binary amounts
02:41 UTC	xrpl-sdk	`6654a7b`	fix(client): paginated calls read one ledger; delivered_amount helpers
02:41 UTC	xrpl-sdk	`fd0b1dc`	fix(utils): reserves in exact drops; exact book liquidity; dex floats labeled
02:41 UTC	xrpl-sdk	`e288ea4`	fix(core): a ledger entry that does not parse is Unparsed, not Other; from_value is strict
02:41 UTC	xrpl-sdk	`c9078f1`	fix(builders): build() checks rippled's rules; AMM mode flags set; AccountTxnID spelled right
02:41 UTC	xrpl-sdk	`a7111f2`	feat(core): keylets for every ledger entry type rippled 3.4.1 has
02:41 UTC	xrpl-sdk	`99290fc`	chore(lints): deny narrowing `as` casts workspace-wide
02:41 UTC	xrpl-sdk	`c0de03c`	docs: 0.6.0: changelog, security policy, CRYPTO.md, parity report; design notes marked historical
02:41 UTC	xrpl-sdk	`ababe1b`	fix(codec): read every JSON currency the same way, "1" (noCurrency) included
02:41 UTC	xrpl-sdk	`a75ea28`	fix(codec): a token amount may use rippled's badCurrency, as rippled's parser allows
02:41 UTC	xrpl-sdk	`52ad1ef`	chore: Cargo.lock for 0.6.0
02:41 UTC	xrpl-sdk	`ec20227`	Merge pull request #33 from ilubxrp589/redteam-fixes
02:41 UTC	xrpl-sdk	`6e69487`	feat(wallet): open a wallet from a BIP-39 phrase, Xaman secret numbers or an RFC 1751 key
02:41 UTC	xrpl-sdk	`73a865a`	fix(client): autofill prices each transaction type as rippled does
02:41 UTC	xrpl-sdk	`a8559a9`	fix(client/wasm): wait with the global setTimeout, not window's
02:41 UTC	xrpl-sdk	`6cd4393`	feat: sign typed transactions, verify signatures, read balances; BlockingClient complete
02:41 UTC	xrpl-sdk	`72e9559`	feat: standard traits on clients and core types

Merged	Repo	Commit	Subject
02:41 UTC	xrpl-sdk	`5712e4d`	feat(client): Client::pages streams pages with no limit
02:41 UTC	xrpl-sdk	`18f7387`	feat(client): HTTP client builder, polite retries, typed HTTP status
02:41 UTC	xrpl-sdk	`5e7e929`	feat(client): XrplWsClient::builder for timeouts, pings and buffers
02:41 UTC	xrpl-sdk	`c3ed958`	docs, ci: contributor guide, issue forms, docs.rs labels, CI on three OSes
02:41 UTC	xrpl-sdk	`df0e574`	feat(core): ledger entry flags as named constants
02:41 UTC	xrpl-sdk	`ad6d7da`	feat: check a ledger against its hash; WS tests that held only on Linux
02:41 UTC	xrpl-sdk	`e26fd0f`	docs: an # Errors section on every public function that can fail
02:41 UTC	xrpl-sdk	`169824e`	release: 0.7.0; fuzz target for key backups and ledger checks
02:41 UTC	xrpl-sdk	`472a78c`	Merge pull request #34 from ilubxrp589/m7-pro
02:41 UTC	xrpl-sdk	`7c12fd4`	license: Apache-2.0 with a NOTICE crediting James Turner
02:41 UTC	xrpl-sdk	`76a11f1`	Merge pull request #35 from ilubxrp589/apache-license
02:41 UTC	xrpl-sdk	`220c123`	feat(client)!: transport errors without reqwest; timeouts are Timeout; caller mistakes are InvalidRequest
02:41 UTC	xrpl-sdk	`d2cd7bb`	feat(client)!: Client = every method on any Transport; Send futures; dyn transports
02:41 UTC	xrpl-sdk	`2b54449`	feat!: protocol types non_exhaustive, built with new + with_ setters
02:41 UTC	xrpl-sdk	`b4bfe6c`	docs(changelog): 0.8 step 3, non_exhaustive types and setters
02:41 UTC	xrpl-sdk	`1c14076`	feat!: the typed models are the one transaction API; builders removed
02:41 UTC	xrpl-sdk	`761b028`	feat!: typed flags per transaction and ledger entry type
02:41 UTC	xrpl-sdk	`dccad87`	chore: versions 0.8.0 (breaking release in progress)
02:41 UTC	xrpl-sdk	`701f8e5`	feat!: one Asset type for models, keylets and the client
02:41 UTC	xrpl-sdk	`2bd6dea`	feat!: one type per idea, and the remaining API polish
02:41 UTC	xrpl-sdk	`af487c8`	feat!: sign with any key: SigningKey and AsyncSigningKey
02:41 UTC	xrpl-sdk	`7723826`	feat!: rippled API version 2 by default, and typed metadata
02:41 UTC	xrpl-sdk	`b1a3ebf`	feat!: order_book and typed path_find
02:41 UTC	xrpl-sdk	`cda0637`	feat!: features that gate code; trimmed dependencies
02:41 UTC	xrpl-sdk	`606d285`	fix: review findings before 0.8.0

Merged	Repo	Commit	Subject
02:41 UTC	xrpl-sdk	`e11b953`	release: 0.8.0
02:41 UTC	xrpl-sdk	`8277ba1`	Merge pull request #36 from ilubxrp589/m8
02:41 UTC	xrpl-sdk	`b3f97d9`	docs: benchmarks re-run for 0.8; counts corrected; every CI job listed
02:41 UTC	xrpl-sdk	`1f79e48`	ci: a weekly check of rippled's definitions on Devnet, Testnet and mainnet
02:41 UTC	xrpl-sdk	`341e037`	perf(core): encode DeliverMax in place; decode addresses with one allocation
02:41 UTC	xrpl-sdk	`676d820`	Merge pull request #37 from ilubxrp589/maint
02:41 UTC	xrpl-sdk	`0296ebe`	docs(core): an example on every public item in address, codec, crypto, error, maybe_send, flags, wasm_bindgen_exports
02:41 UTC	xrpl-sdk	`352e160`	docs(codec): drop a stray doc comment from a private constant
02:41 UTC	xrpl-sdk	`68977cc`	docs(client, sdk): an example on every public item in the client's client, blocking, ws, http, stream, faucet and error modules and the SDK
02:41 UTC	xrpl-sdk	`d3ec352`	docs(types): an example on every public item in types/, hashes.rs, ledger.rs, transaction/{batch,cost,metadata,mod}.rs and utils/
02:41 UTC	xrpl-sdk	`0cb6b60`	fix(client): ManifestEvent keeps the serialized manifest
02:41 UTC	xrpl-sdk	`dd60034`	fix(codec): TransactionResult by name, as rippled writes and reads it
02:41 UTC	xrpl-sdk	`8a7d0e4`	docs(client): an example on every public item in requests.rs and types/
02:41 UTC	xrpl-sdk	`fe1ee9b`	docs(client): six response docs that disagreed with rippled's answers
02:41 UTC	xrpl-sdk	`84ba2e8`	docs(models): an example on every public item in transaction/models/ and ledger_object/formats.rs
02:41 UTC	xrpl-sdk	`0f70bdd`	chore: stop tracking Python bytecode (**pycache**)
02:41 UTC	xrpl-sdk	`0d056ba`	fix(models): Batch::validate runs rippled's ledger-free Batch checks
02:41 UTC	xrpl-sdk	`8144d21`	style: rustfmt
02:41 UTC	xrpl-sdk	`73572fb`	fix(core): no unused import in a no_std build
02:41 UTC	xrpl-sdk	`79859c5`	ci: warnings fail the MSRV / no_std job
02:41 UTC	xrpl-sdk	`8b79f6c`	docs: changelog and roadmap for the examples pass
02:41 UTC	xrpl-sdk	`3c06631`	Merge pull request #38 from ilubxrp589/docs-examples
02:41 UTC	xrpl-sdk	`a205e5e`	release: 0.8.1
02:41 UTC	xrpl-sdk	`9279f39`	Merge pull request #39 from ilubxrp589/release-0.8.1
02:41 UTC	xrpl-sdk	`4fdc498`	Merge pull request #32 from ilubxrp589/m5-trust

Merged	Repo	Commit	Subject
02:41 UTC	xrpl-sdk	`86cadcb`	feat: 0.5.0, trust: fuzzing, benchmarks, MSRV, rippled parity at the edges
02:41 UTC	xrpl-sdk	`5f587bf`	Merge pull request #31 from ilubxrp589/m4-dx
02:41 UTC	xrpl-sdk	`ef5e034`	feat: cookbook, Amount helpers, every public item documented; 0.4.0
02:41 UTC	xrpl-sdk	`dac53ff`	feat: hxrpl, the command line (new crate halcyon-xrpl-cli)
02:41 UTC	xrpl-sdk	`3864bfa`	feat(client): BlockingClient — every Client method as a plain call
02:41 UTC	xrpl-sdk	`a68a422`	feat(client): typed WebSocket stream events for every rippled stream
02:41 UTC	xrpl-sdk	`eb2c287`	Merge pull request #30 from ilubxrp589/m3-transactions
02:41 UTC	xrpl-sdk	`3f39e24`	feat: typed models for all 82 transaction types, and Batch (XLS-56); 0.3.0
02:41 UTC	xrpl-sdk	`07eab00`	Merge pull request #29 from ilubxrp589/post-release-0.2.0
02:41 UTC	xrpl-sdk	`d98c5d7`	docs: 0.2.0 is out — CHANGELOG, ROADMAP, README badges
02:41 UTC	xrpl-sdk	`a0a6f6c`	Merge pull request #28 from ilubxrp589/m2-requests
02:41 UTC	xrpl-sdk	`1aed483`	feat(client): a typed request and response for every public rippled method
02:41 UTC	xrpl-sdk	`c9ccbfe`	Merge pull request #27 from ilubxrp589/release-0.2.0
02:41 UTC	xrpl-sdk	`2890e63`	release: 0.2.0, the first crates.io release
02:41 UTC	xrpl-sdk	`c38fa00`	Merge pull request #26 from ilubxrp589/m1-helpers
02:41 UTC	xrpl-sdk	`79f4949`	feat: faucet, Wallet::seed, live Testnet test; fix fee and UInt64; full docs
02:41 UTC	xrpl-sdk	`42fe58a`	feat(core): transaction hashes, ledger entry IDs, channel claims, everyday utils
02:41 UTC	xrpl-sdk	`004ee04`	Merge pull request #25 from ilubxrp589/deps-update
02:41 UTC	xrpl-sdk	`64541ff`	Merge pull request #24 from ilubxrp589/sdk-next
02:41 UTC	xrpl-sdk	`a9efcd5`	deps: cargo update; actions/checkout v4 -> v7
02:41 UTC	xrpl-sdk	`ebe99c6`	release prep: publish as halcyon-xrpl-*, tel results, drop the stale field table
02:41 UTC	xrpl-sdk	`b233b21`	docs: CHANGELOG and README for the Client trait, typed objects and amounts
02:41 UTC	xrpl-sdk	`9bf6d7b`	feat: typed ledger objects and typed amounts (MPT included)
02:41 UTC	xrpl-sdk	`5073f6e`	feat(client): one Client trait over HTTP, WS and wasm WS; typed requests
02:41 UTC	xrpl-sdk	`9252046`	fix(wasm): the wasm build compiles again; CI checks it

Merged	Repo	Commit	Subject
02:41 UTC	xrpl-sdk	`a2a1464`	docs(readme): current numbers — 34/34 + 261/261 fixtures, 28 builders
02:41 UTC	xrpl-sdk	`6e6b6fd`	chore: enforce the safety rules as workspace lints
02:41 UTC	xrpl-sdk	`eaece20`	feat(codec): rippled 3.4.1 parity — Int32, PermissionValue names, X-addresses
02:41 UTC	xrpl-sdk	`2853e26`	chore(deps): bump rustls, rustls-webpki, quinn-proto, webpki-roots, rand
02:41 UTC	xrpl-sdk	`4b923d8`	chore: list the public xrpl.js test seeds in .push-guard-allow
02:41 UTC	xrpl-sdk	`f1f1722`	docs: correct TESTING.md's known-address vectors; CHANGELOG Unreleased
02:41 UTC	xrpl-sdk	`bdb2a59`	fix(core): Ed25519 signs the signing data itself, as rippled verifies it
02:41 UTC	xrpl-sdk	`01e823b`	chore(core): clear the clippy and rustfmt failures
02:41 UTC	xrpl-sdk	`24f3af2`	fix(sdk): order multisign Signers by AccountID bytes, not base58 text

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **fix(core): the encoder refuses what rippled refuses instead of changing the transaction** (`55fb80b`, xrpl-sdk)
- A field name the definitions do not hold is CoreError::UnknownField, at the top level and inside inner objects and arrays. Before, it was skipped, so a misspelt DestinationTag ("destination_tag", "DestinationTAg") vanished from the signed transaction. rippled's response-only keys (hash, meta, ledger_index, ...) are still skipped at the top level. - UInt8/UInt16/UInt32 values that do not fit are refused. Before, …
- **fix(core): token values parse and round exactly as rippled reads them** (`37dc627`, xrpl-sdk) IouAmount::from_decimal cast a u64 mantissa to i64, so a value whose digits reach 2^63 flipped sign ("9999999999999999999" became -8446744073709551000). It also read "", "-", "." and "e5" as zero, wrapped long inputs through an i16 exponent, panicked on exponent overflow in debug builds, truncated past 16 digits where rippled rounds, and errored below 10^-81 where rippled gives zero.
- **fix(client): submit_and_wait answers only with final outcomes; autofill caps the fee** (`1ef2462`, xrpl-sdk) submit_and_wait could report a transaction that went through as failed or expired, and a validated failure as Ok. Each of those invites a caller to sign the payment again: - a preliminary tec was returned as the outcome; - terQUEUED was resubmitted, and the resubmit's tef reported as "never reaches a ledger"; - expiry was decided from the open ledger before the transaction was looked up, so one included in its last …
- **fix(crypto): secrets stay out of logs and errors; secp256k1 signs only a 32-byte hash** (`1a10b5a`, xrpl-sdk)
- secp256k1 sign/verify took any input and the ECDSA primitive used only its first 32 bytes, so Keypair::sign/verify on a secp256k1 key signed a long message's prefix: the signature verified for any message sharing it. Both now refuse anything but a 32-byte hash. New Keypair::sign_message / verify_message sign as rippled verifies (Ed25519 over the message, secp256k1 over SHA-512Half), the same for both key types; …

- **fix(cli): the server decides whether --yes is needed; failed sends exit non-zero** (`b5fd628`, xrpl-sdk)
- The mainnet guard checked only --network (default testnet), so `--url`/`XRPL_URL` pointing at a mainnet server sent real XRP without --yes, and `raw submit` had no guard at all. Before submitting, hxrpl now asks the server its network_id: only Testnet (1) and Devnet (2) go ahead without --yes; mainnet, other networks (Xahau, sidechains) and servers that do not say need it. `--network mainnet` still refuses …
- **fix(client): the WebSocket client closes on drop, notices dead connections, and routes each subscription its own events** (`cb9c67c`, xrpl-sdk) WebSocket (native): - One supervisor task owns the connection. Dropping the client (or close()) stops it and closes the socket; before, the reconnect task kept the socket, keepalive and resubscriptions alive forever. - A connection silent for 45 s (pings every 15 s) is replaced: a laptop that slept or a NAT that dropped the connection no longer leaves the client "connected" with streams quiet and requests timing …
- **fix(transaction): Batch checks what rippled checks; a transaction that does not parse is Unparsed, not Other** (`58116da`, xrpl-sdk)
- Tx: a transaction of a known type that does not parse leniently (inside a stream event or response) is now Tx::Unparsed { transaction_type, error, value } instead of Tx::Other, which is kept for unknown types. A Payment with a field a newer rippled added no longer looks like an unknown transaction to code matching Tx::Payment; the client logs it. - Tx::from_value is strict for a Batch's inner transactions too …
- **fix(core): canonical currencies, minimal field headers, capped base58, checked binary amounts** (`eafb7ae`, xrpl-sdk)
- Currency is canonical: from_hex of a standard code is the standard code (Eq/Hash agree with from_ascii, and it prints "USD" as rippled does); "XRP" is XRP's all-zero code (rippled's to_currency); rippled's noCurrency reads and prints as "1". New Currency::parse (rippled's JSON forms) and Currency::from_code (also 4-20 character codes such as "RLUSD"), used by Amount::token and the keylets (keylet::trust_line took …
- **fix(client): paginated calls read one ledger; delivered_amount helpers** (`6654a7b`, xrpl-sdk)
- request_all (account_lines_all, account_objects_all, ledger_data, …) pinned only the marker: each page re-resolved "validated", so a result could mix two ledgers (missing or doubled trust lines in a total). Every page after the first now asks for the first page's ledger (Paginated::ledger_of / at_ledger; account_tx and nft_history, which page across ledgers, are unchanged). - utils::delivered_amount(meta), …
- **fix(utils): reserves in exact drops; exact book liquidity; dex floats labeled** (`fd0b1dc`, xrpl-sdk) The utils module promised "all exact, never floating point", but: - reserve.rs multiplied XRP floats: (reserve_xrp * 1e6) as u64 turns 1.001 into 1,000,999 drops, NaN or a negative reserve into 0 (so the whole balance looked spendable), and owner_count could overflow. Now: total_reserve_drops / available_balance_drops take drops (u64, saturating), and reserve_xrp_to_drops converts server_info's XRP value exactly …
- **fix(core): a ledger entry that does not parse is Unparsed, not Other; from_value is strict** (`e288ea4`, xrpl-sdk)
- LedgerObject deserialization (inside responses: account_objects, ledger_data, ledger_entry) turned any entry that did not fit its type into LedgerObject::Other, so code matching LedgerObject::RippleState silently missed it. It is now LedgerObject::Unparsed { entry_type, error, value }; Other is kept for unknown entry types. - LedgerObject::from_value is strict: a field the type's struct does not have (it would be …
- **fix(builders): build() checks rippled's rules; AMM mode flags set; AccountTxnID spelled right** (`c9078f1`, xrpl-sdk)

- TxCommon serialized account_txn_id as "AccountTxnId" (serde PascalCase), a name rippled does not have: the old encoder dropped it silently, so the AccountTxnID guard never reached the ledger. Now "AccountTxnID" (also in the deprecated CommonFields). New TxCommon fields: network_id ("NetworkID") and delegate. - AMMDeposit / AMMWithdraw set no mode flag, which rippled requires (exactly one): build() now infers it …
- **feat(core): keylets for every ledger entry type rippled 3.4.1 has** (`a7111f2`, xrpl-sdk) The keylet module said "every kind of ledger entry" but had no way to find these; now it does, each checked against rippled 3.4.1's own answer (ledger_entry reports the index it computes even for a missing entry): - bridge (from either door), xchain_owned_claim_id, xchain_owned_create_account_claim_id (Bridge describes both chains); - deposit_preauth_credentials (credentials sorted and de-duplicated as rippled's …
- **fix(codec): read every JSON currency the same way, "1" (noCurrency) included** (`ababe1b`, xrpl-sdk) Found by fuzz_roundtrip: since currencies print as rippled prints them, a token amount (or Issue, or path step) in rippled's noCurrency decodes as currency "1", which the token-amount encoder did not read back. Every JSON currency now goes through Currency::parse ("XRP", "1", ISO code, 40 hex digits), as encode_currency_type already did for Oracle assets.
- **fix(codec): a token amount may use rippled's badCurrency, as rippled's parser allows** (`a75ea28`, xrpl-sdk) Found by fuzz_roundtrip: the encoder refused a token whose 40-hex currency is "XRP" in the ISO position (badCurrency), which rippled's binary reader and JSON parser both take (only its preflight refuses it, with temBAD_CURRENCY), so a decoded transaction could fail to encode again. Only XRP's own code (all zero, or "XRP") is refused now, as the parser refuses it.
- **feat(wallet): open a wallet from a BIP-39 phrase, Xaman secret numbers or an RFC 1751 key** (`6e69487`, xrpl-sdk) Wallet::from_mnemonic / from_mnemonic_with (BIP-39 + BIP-32, m/44'/144'/0'/0/0 by default, as Ledger and xrpl.js), from_secret_numbers (XLS-12, secp256k1 as Xaman) and from_rfc1751 (rippled's wallet_propose master_key). In xrpl_core: crypto::{mnemonic, hd, secret_numbers, rfc1751}, Seed and Keypair constructors, CoreError::InvalidMnemonic / InvalidDerivationPath.
- **fix(client): autofill prices each transaction type as rippled does** (`73a865a`, xrpl-sdk) autofill charged every type one base fee, so AccountDelete and AMMCreate (one owner reserve: 0.2 XRP on mainnet) were refused with telCAN_NOT_QUEUE and submit_and_wait resubmitted them until they expired; EscrowFinish with a fulfillment and the Confidential MPT types too. New xrpl_core::transaction::cost::transaction_cost follows rippled 3.4.1's calculateBaseFee (owner reserve types, 32 + n/16 for a fulfillment, +9 …
- **fix(client/wasm): wait with the global setTimeout, not window's** (`a8559a9`, xrpl-sdk) sleep_secs set its timer only through web_sys::window(), so in a Web Worker, Service Worker, Node.js or Cloudflare Worker (no window) submit_and_wait, fund_and_wait and the browser WebSocket request timeout waited forever. It now calls setTimeout on js_sys::global(). A wasm unit test checks a 1 s wait completes in Node.js (no window); a CI step runs the client's wasm tests there with the wasm-bindgen-test-runner …
- **feat: sign typed transactions, verify signatures, read balances; BlockingClient complete** (`6cd4393`, xrpl-sdk)
- Wallet::sign(&tx) takes a typed model, a models::Tx, a builder (validated first) or JSON (one trait, transaction::ToTxJson) and returns a SignedTransaction { tx_json, tx_blob, hash }; sign_and_submit(&client, &tx, &wallet) autofills a copy, signs, submits and waits. README quick start and testnet_quickstart use a typed Payment (run live on Testnet). - verify_transaction(&tx): single, multi-signed (account order) …
- **feat: standard traits on clients and core types** (`72e9559`, xrpl-sdk) XrplHttpClient: Clone. HTTP, WebSocket (native + wasm) and blocking clients: Debug showing the host only (a test checks credentials, path and query never show). Subscription::into_stream (futures::Stream, native and wasm). Hash128/160/192/256: FromStr, Ord,

AsRef<[u8]>. AccountId: Ord (byte order = rippled's signer order), AsRef<[u8]>. KeyType: Display, FromStr, Hash, serde as rippled's key_type. …

- **feat(client): Client::pages streams pages with no limit** (`5712e4d` , xrpl-sdk) request_all stops at MAX_PAGES (50) and returns only an error, so a long account_tx could not be read. Client::pages(req) is a futures::Stream of pages that follows the marker with no limit, each page from the first one's ledger; BlockingClient::pages is an iterator. A test reads 60 pages on one ledger. PaginationLimitReached's message now names Client::pages.
- **feat(client): HTTP client builder, polite retries, typed HTTP status** (`18f7387` , xrpl-sdk) XrplHttpClient::builder(url): timeout, connect timeout, headers (Debug shows names only; values marked sensitive), user agent, fee cap and a RetryPolicy. A request is retried only when the server says it is busy and did not take it: HTTP 429/503 or rippled slowDown/tooBusy, waiting Retry-After (capped at 10 s) or 1 s, 2 s; never after a timeout or a dropped connection, so a retry cannot send a transaction twice. …
- **feat(client): XrplWsClient::builder for timeouts, pings and buffers** (`5e7e929` , xrpl-sdk) The WebSocket client's request timeout (30 s), connect timeout (15 s), ping interval (15 s), silence limit (45 s), event buffer (1,024) and fee cap were constants; XrplWsClient::builder(url) sets each (Debug shows the host only). connect(url) keeps the defaults. Tests: a set 300 ms request timeout fires against a server that never answers; a subscription read through into_stream.
- **feat(core): ledger entry flags as named constants** (`df0e574` , xrpl-sdk) gen_ledger_objects.py now reads server_definitions' LEDGER_ENTRY_FLAGS: each typed ledger entry with flags gets its LSF_ constants (51, documented), ALL_FLAGS, and has_flag(). A test checks every flag bit on the mainnet AccountRoot, RippleState and Offer samples is a named one.
- **feat: check a ledger against its hash; WS tests that held only on Linux** (`ad6d7da` , xrpl-sdk)
- xrpl_core::ledger: LedgerHeader (bytes, rippled JSON) and hash(); transaction_tree_hash / state_tree_hash (SHAMap roots: SND/MLN leaves, MIN inner, root always inner, empty = zero); verify_ledger_response for a binary `ledger` with expanded transactions. Client::verified_ledger (and BlockingClient) fetches and checks one. Tested on mainnet ledger 107852951 (34 txs, recorded from rippled 3.4.1): header JSON hashes …
- **feat(client)!: transport errors without reqwest; timeouts are Timeout; caller mistakes are InvalidRequest** (`220c123` , xrpl-sdk) 0.8 step 1 (errors). ClientError::Http(reqwest::Error) is replaced by Transport { kind: TransportErrorKind, message } (connect, request, body, redirect, setup, other; message = the whole chain, no URL), so reqwest leaves the public API. An HTTP timeout is ClientError::Timeout(configured duration), as on WebSocket. Caller mistakes (no Account, not an object, a malformed Batch, a bad header) are the new …
- **feat(client)!: Client = every method on any Transport; Send futures; dyn transports** (`d2cd7bb` , xrpl-sdk) 0.8 step 2. Transport is object safe: send_raw(&self, method, params) -> TransportFuture (boxed, Send on native) and max_fee_drops; implemented for &T, Box, Arc. Client: Transport has every typed method as `fn x(..) -> impl Future<Output = ..> + MaybeSend` with a blanket impl for every Transport, so Arc is a client and generic code can spawn the futures (MaybeSend/MaybeSync = Send/Sync on …
- **feat!: protocol types non_exhaustive, built with new + with_ setters** (`2b54449` , xrpl-sdk) 0.8 step 3. #[non_exhaustive] on the 82 transaction models (generator), their nested types, Common, Tx, the 31 ledger entry types (generator), LedgerObject, all 43 requests and 87 response types, so a field rippled adds later is not breaking. Construction: new(required) + with_ setters: 98 generated on requests (TxRequest::by_ctid added; parameterless requests get new()), 179 generated on transaction models, and …
- **feat!: the typed models are the one transaction API; builders removed** (`1c14076` , xrpl-sdk) 0.8 step 4. transaction::builders (28 types, Transaction trait, TxCommon, xrpl_sdk::transactions) and the deprecated

CommonFields / TransactionType are removed. Their preflight checks are ported to the models with typed fields: TxModel::validate (default Ok; generated overrides for AccountSet, AMMCreate/Deposit/Withdraw/Vote/Bid, Clawback, DelegateSet, SignerListSet, TrustSet, NFTokenAcceptOffer, in …

- **feat!: typed flags per transaction and ledger entry type** (`761b028`, xrpl-sdk) 0.8 step 5. xrpl_core::flags: the Flags trait (ALL, from_bits, bits, contains, is_empty, unknown_bits) and flags_type!, which defines a transparent u32 newtype with named consts, |, &, -, |=, From<_> for u32 and a Debug listing names. Generated: Flags for the 23 transaction types with flags (each also FULLY_CANONICAL_SIG and INNER_BATCH_TXN), UniversalFlags for the rest, TxModel::Flags with typed …
- **feat!: one Asset type for models, keylets and the client** (`701f8e5`, xrpl-sdk) 0.8 step 6a. xrpl_core::types::Asset enum (Xrp, Token, Mpt): validated constructors, rippled's JSON (strict: no issuer on XRP, issuer required on a token, no extra keys), rippled's ordering (MPT < XRP/token; currency then issuer), Display/FromStr (XRP, USD.r…, MPT:), Amount::asset(). Replaces models::Asset (now a re-export), keylet::Issue/Bridge and the Value params of book_offers / amm_info / BookOffersRequest …
- **feat!: one type per idea, and the remaining API polish** (`2bd6dea`, xrpl-sdk)
- PathStep: the client's is the transaction models' (a found path goes into Payment::paths as it is); the client's AuthAccount is now AuctionSlotAccount. - AccountData::balance is an Amount. - Wallet::sign_for_multisigning returns a typed models::Signer; collect_signers takes them. - LedgerIndex is passed by value everywhere; as_value -> to_value. - account_tx / account_tx_all take only the account; options go …
- **feat!: sign with any key: SigningKey and AsyncSigningKey** (`af487c8`, xrpl-sdk) A key that never leaves an HSM, a cloud KMS or a hardware wallet can now sign. xrpl_core::crypto::SigningKey (synchronous) and AsyncSigningKey (remote, async fn) describe a key; Keypair and Wallet implement both, and every signing function takes any key: sign_transaction (+ _async), sign_for_multisigning (+ _async, for a named account), batch and channel-claim signing, and the SDK's sign / sign_async / …
- **feat!: rippled API version 2 by default, and typed metadata** (`7723826`, xrpl-sdk) Every request (and subscription) asks for API version 2 unless its parameters name one (xrpl_client::API_VERSION), and the response types follow version 2 only: - One TxResult for a transaction with its metadata, whichever method returns it (tx, transaction_entry, account_tx, nft_history, expanded ledger as LedgerTx): tx_json with hash, ledger_index, ledger_hash, close_time_iso, ctid, validated beside it; tx_blob / …
- **feat!: order_book and typed path_find** (`b1a3ebf`, xrpl-sdk)
- Client::order_book(base, quote, limit): both sides of a book from one validated ledger (asks lowest first, bids highest first) as an OrderBook with best_ask / best_bid / spread / midpoint; BlockingClient too. Amount::to_f64 for display prices. The dex_order example uses it (run on Testnet). - XrplWsClient::path_find(&PathFindRequest) -> (PathFindResult, Subscription) and path_find_close; WsEvent::PathFind is …
- **feat!: features that gate code; trimmed dependencies** (`cda0637`, xrpl-sdk)
- halcyon-xrpl-client / -sdk: `http` (XrplHttpClient, faucet, BlockingClient) and `ws` (XrplWsClient) now decide what is built; both on by default. Without either, the client is the Client trait and the typed requests/responses. The SDK gains a `wasm` feature and builds for wasm32. `full` and `live-tests` are gone; the live tests run with RUSTFLAGS="--cfg xrpl_live_tests" (checked by `unexpected_cfgs`). - tokio: no …
- **fix: review findings before 0.8.0** (`606d285`, xrpl-sdk)
- autofill of a Batch sent with a Ticket numbered the outer account's inner transactions from 1 (tefPAST_SEQ while the Batch succeeded); they now start at the account's sequence. Test added. - Client::raw_request adds

api_version, so a Transport of your own gets API version 2 too; Client documents that its waits use Tokio's timer. - get_nftoken_id's page diff counts only pages whose NFTokens changed (a page whose …

- **perf(core): encode DeliverMax in place; decode addresses with one allocation** (`341e037` , xrpl-sdk)
- Since 0.6 the encoder copied the whole transaction to drop or rename a Payment's DeliverMax. It now checks it as before (alone it is the Amount, beside an equal Amount it is left out, otherwise an error) and writes it while encoding: a mainnet Payment encodes in 3.84 µs (4.64 µs in 0.8.0, 3.88 µs before 0.6). - base58 decoding sizes its buffer from the input instead of growing it: a classic address decodes in 373 …
- **fix(client): ManifestEvent keeps the serialized manifest** (`0cb6b60` , xrpl-sdk) rippled's pubManifest sends `manifest` (the serialized manifest, hex) with every manifestReceived message; the type dropped it. Added as an optional field (additive: the struct is non_exhaustive). A test checks the event against a real validator's manifest (tests/fixtures/rippled/manifest.json) in the message shape rippled builds, keeping every field; the module docs say so. The path_find example names real …
- **fix(codec): TransactionResult by name, as rippled writes and reads it** (`dd60034` , xrpl-sdk) rippled's STUInt8::getJson writes sfTransactionResult as the result's name (tesSUCCESS) and STParsedJSON reads a name whose code fits the byte (tes, tec). The decoder wrote the number, so metadata decoded from bytes did not read as TransactionMetadata; the encoder refused the name. Both now follow rippled; codec::definitions gains transaction_result_code / _name. Tests: names round trip, other names are refused, …
- **fix(models): Batch::validate runs rippled's ledger-free Batch checks** (`0d056ba` , xrpl-sdk) Exactly one mode flag; 2 to 8 inner transactions (rippled: temARRAY_EMPTY at preflight, more than 8 refused by STTx); no Batch inside a Batch (temINVALID_INNER_BATCH); no inner transaction with both a Sequence and a TicketSequence (temSEQ_AND_TICKET); each inner transaction's own checks. Only rules both rippled 3.1.2 and 3.3.0 sources hold (the disabled inner types and the BatchSigners cap differ between versions, …
- **fix(core): no unused import in a no_std build** (`73572fb` , xrpl-sdk) The base58 decoder no longer uses vec!, so alloc::vec was unused without std (a warning since PR #37).
- **feat: 0.5.0, trust: fuzzing, benchmarks, MSRV, rippled parity at the edges** (`86cadcb` , xrpl-sdk) Fuzzing in CI (8 targets, 4 new, seeded from real XRPL data), benchmarks against xrpl-rust 1.3.0 on the same inputs, rust-version (core 1.85, the rest 1.88) checked in CI, and docs/PARITY.md listing every check against rippled 3.4.1.
- **feat: cookbook, Amount helpers, every public item documented; 0.4.0** (`ef5e034` , xrpl-sdk)
- Cookbook: 12 runnable recipes in crates/xrpl-sdk/examples, each with its own faucet wallets (issue_token, nft_marketplace, dex_order, amm, escrow, checks, payment_channel, multisign, batch, mpt, blocking, watch_streams), all run end to end on Testnet (watch_streams on mainnet) before release; they replace send_payment / mint_nft / place_offer. - Amount::token (long codes become hex), Amount::mpt, and Display ("25 …
- **feat: hxrpl, the command line (new crate halcyon-xrpl-cli)** (`dac53ff` , xrpl-sdk) wallet new/show/faucet; account info/lines/offers/objects/nfts/tx/ currencies/channels; tx get/hash/decode/encode/sign/simulate/submit/ send-xrp; ledger, entry, id (ledger entry IDs), fee, server, book, watch (live streams), convert (XRP/drops, ledger time, NFTokenIDs, hex), raw. Testnet by default; sending on mainnet needs --yes; seeds only from XRPL_SEED or --seed-file. Long currency codes (RLUSD) become hex. …
- **feat(client): BlockingClient — every Client method as a plain call** (`3864bfa` , xrpl-sdk) A single-threaded Tokio runtime owned by the client runs each call to completion: request / request_all / raw_request, account_info, fee, ledger, tx, autofill, simulate, submit, submit_multisigned, submit_and_wait, and run(|c| …) / block_on for anything else. Re-exported from xrpl_sdk.

- **feat(client): typed WebSocket stream events for every rippled stream** (`a68a422` , xrpl-sdk) WsEvent gains Validation, Server, Consensus, BookChanges, Manifest, PeerStatus, PathFind and Other (non_exhaustive); TransactionEvent carries a typed Tx. XrplWsClient::subscribe / unsubscribe for any parameters, plus subscribe_validations / server / consensus / book_changes / manifests / transactions_proposed / order_book (with a typed OrderBookSnapshot). Checked against messages recorded from rippled 3.4.1 on …
- **feat: typed models for all 82 transaction types, and Batch (XLS-56); 0.3.0** (`3f39e24` , xrpl-sdk)
- transaction::models: a struct per transaction type, generated from rippled 3.4.1's server_definitions (scripts/gen_transactions.py): Common fields, each type's own fields (334), `new` with the required ones, flags as constants (100). Typed nested arrays (Memo, Signer, SignerEntry, PriceData, Permission, CredentialSpec, BatchSigner, inner Tx), Asset, XChainBridge, PathStep. `Tx` over all: strict `from_value` , …
- **feat(client): a typed request and response for every public rippled method** (`1aed483` , xrpl-sdk)
- requests: 43 typed requests (25 new), grouped as on xrpl.org: account, ledger, transaction, path and order book, payment channel, server info, utility, vault, and Clio's nft_info / nft_history / nfts_by_issuer / mpt_holders. Each has `new` with its required parameters and every optional rippled parameter as a field; the paged ones are `Paginated` . - types: split by method group; every response models all the …
- **feat: faucet, Wallet::seed, live Testnet test; fix fee and UInt64; full docs** (`79f4949` , xrpl-sdk)
- xrpl-client: faucet (Testnet / Devnet) with fund / fund_and_wait; FaucetNetwork knows each network's public servers. - xrpl-sdk: generate_faucet_wallet, Wallet::seed (a generated or restored wallet keeps its seed), Wallet::sign_payment_channel_claim; re-exports of hashes, keylet, utils, channel claims, and both crates in full. - tests/live_testnet.rs (--features live-tests): two faucet wallets create 20 kinds of …
- **feat(core): transaction hashes, ledger entry IDs, channel claims, everyday utils** (`42fe58a` , xrpl-sdk)
- hashes: transaction_id / hash_signed_tx / hash_signed_tx_blob (refuses an unsigned tx unless pseudo or a Batch inner tx), and keylet::* for every ledger entry type rippled 3.4.1 has (Indexes.cpp namespaces). - crypto::channel: sign / verify payment channel claims. - utils: xrp_to_drops / drops_to_xrp, ledger time <-> Unix / ISO 8601 / SystemTime, str_to_hex / hex_to_str, TransferRate and Quality conversions, …
- **feat: typed ledger objects and typed amounts (MPT included)** (`9bf6d7b` , xrpl-sdk) Ledger objects: types::ledger_object has one struct per ledger entry type rippled 3.4.1 defines (31), generated from its server_definitions LEDGER_ENTRY_FORMATS by scripts/gen_ledger_objects.py — required fields plain, optional/default ones Option, fields typed (AccountId, Amount, Hash128..256, u8..u32, i32), UInt64/Blob/Number as rippled's strings, inner objects/arrays as JSON. LedgerObject wraps them by …
- **feat(client): one Client trait over HTTP, WS and wasm WS; typed requests** (`5073f6e` , xrpl-sdk) http.rs, ws.rs and ws_wasm.rs each carried their own copy of every RPC method (and they had drifted: WS submit never sorted engine results, WS submit_and_wait never resubmitted on ter, the wasm client had only five methods). Now a transport implements Client::raw_request and the trait provides the rest once: the 18 RPC methods, autofill, submit_and_wait and the _all paginators.
- **fix(wasm): the wasm build compiles again; CI checks it** (`9252046` , xrpl-sdk) rand 0.9 draws on getrandom 0.3, which on wasm32 needs its wasm_js feature and the getrandom_backend="wasm_js" cfg. Neither was set, so `cargo check --target wasm32-unknown-unknown -p xrpl-client --features wasm` failed in getrandom — and nothing ran it. xrpl-core's `wasm` feature now enables getrandom 0.3's wasm_js, and .cargo/config.toml sets the cfg for this workspace (downstream wasm builds set it too).
- **feat(codec): rippled 3.4.1 parity — Int32, PermissionValue names, X-addresses** (`eaece20` , xrpl-sdk)

- definitions.json is now rippled 3.4.1's server_definitions (the reference node's xrpld): +44 fields (Sponsorship, Confidential MPT, LoanScale's sibling RemainingOwnerCountDelta, ...), +7 tx types, +1 ledger entry type, +8 results; Hooks-only fields dropped; UInt32 nth 53 is ImmutableFlags (was MutableFlags); types Hash384/Hash512. - Int32 (STInteger): big-endian two's complement; JSON is the number, and a decimal string …
- **fix(core): Ed25519 signs the signing data itself, as rippled verifies it** (`bdb2a59` , xrpl-sdk) rippled verifies secp256k1 over SHA512Half(signing data) but Ed25519 over the signing data itself (PublicKey.cpp `verify`). sign_transaction and Wallet::sign_for_multisigning hashed first for both key types, so every Ed25519 TxnSignature failed rippled's check — and Wallet::generate defaults to Ed25519.
- **fix(sdk): order multisign Signers by AccountID bytes, not base58 text** (`24f3af2` , xrpl-sdk) rippled (STTx.cpp, "Signers must be in sorted order by AccountID") rejects "Unsorted Signers array" unless signers ascend by their 20-byte AccountID. collect_signers sorted the base58 strings, and the XRPL alphabet is not in ASCII order, so e.g. rPT1Sjq2... sorted before rw2ciyaN... although its AccountID is the larger one.

Docs and operations

- **chore(lints): deny narrowing `as` casts workspace-wide** (`99290fc` , xrpl-sdk)
- **docs: 0.6.0: changelog, security policy, CRYPTO.md, parity report; design notes marked historical** (`c0de03c` , xrpl-sdk)
- **chore: Cargo.lock for 0.6.0** (`52ad1ef` , xrpl-sdk)
- **Merge pull request #33 from ilubxrp589/redteam-fixes** (`ec20227` , xrpl-sdk)
- **docs, ci: contributor guide, issue forms, docs.rs labels, CI on three OSes** (`c3ed958` , xrpl-sdk)
- **docs: an # Errors section on every public function that can fail** (`e26fd0f` , xrpl-sdk)
- **release: 0.7.0; fuzz target for key backups and ledger checks** (`169824e` , xrpl-sdk)
- **Merge pull request #34 from ilubxrp589/m7-pro** (`472a78c` , xrpl-sdk)
- **license: Apache-2.0 with a NOTICE crediting James Turner** (`7c12fd4` , xrpl-sdk)
- **Merge pull request #35 from ilubxrp589/apache-license** (`76a11f1` , xrpl-sdk)
- **docs(changelog): 0.8 step 3, non_exhaustive types and setters** (`b4bfe6c` , xrpl-sdk)
- **chore: versions 0.8.0 (breaking release in progress)** (`dccad87` , xrpl-sdk)
- **release: 0.8.0** (`e11b953` , xrpl-sdk)
- **Merge pull request #36 from ilubxrp589/m8** (`8277ba1` , xrpl-sdk)
- **docs: benchmarks re-run for 0.8; counts corrected; every CI job listed** (`b3f97d9` , xrpl-sdk)
- **ci: a weekly check of rippled's definitions on Devnet, Testnet and mainnet** (`1f79e48` , xrpl-sdk)
- **Merge pull request #37 from ilubxrp589/maint** (`676d820` , xrpl-sdk)
- **docs(core): an example on every public item in address, codec, crypto, error, maybe_send, flags, wasm_bindgen_exports** (`0296ebe` , xrpl-sdk)
- **docs(codec): drop a stray doc comment from a private constant** (`352e160` , xrpl-sdk)
- **docs(client, sdk): an example on every public item in the client's client, blocking, ws, http, stream, faucet and error modules and the SDK** (`68977cc` , xrpl-sdk)
- **docs(types): an example on every public item in types/, hashes.rs, ledger.rs, transaction/{batch,cost,metadata,mod}.rs and utils/** (`d3ec352` , xrpl-sdk)

- **docs(client): an example on every public item in requests.rs and types/** (`8a7d0e4` , xrpl-sdk)
- **docs(client): six response docs that disagreed with rippled's answers** (`fe1ee9b` , xrpl-sdk)
- **docs(models): an example on every public item in transaction/models/ and ledger_object/formats.rs** (`84ba2e8` , xrpl-sdk)
- **chore: stop tracking Python bytecode (pycache)** (`0f70bdd` , xrpl-sdk)
- **style: rustfmt** (`8144d21` , xrpl-sdk)
- **ci: warnings fail the MSRV / no_std job** (`79859c5` , xrpl-sdk)
- **docs: changelog and roadmap for the examples pass** (`8b79f6c` , xrpl-sdk)
- **Merge pull request #38 from ilubxrp589/docs-examples** (`3c06631` , xrpl-sdk)
- **release: 0.8.1** (`a205e5e` , xrpl-sdk)
- **Merge pull request #39 from ilubxrp589/release-0.8.1** (`9279f39` , xrpl-sdk)
- **Merge pull request #32 from ilubxrp589/m5-trust** (`4fdc498` , xrpl-sdk)
- **Merge pull request #31 from ilubxrp589/m4-dx** (`5f587bf` , xrpl-sdk)
- **Merge pull request #30 from ilubxrp589/m3-transactions** (`eb2c287` , xrpl-sdk)
- **Merge pull request #29 from ilubxrp589/post-release-0.2.0** (`07eab00` , xrpl-sdk)
- **docs: 0.2.0 is out — CHANGELOG, ROADMAP, README badges** (`d98c5d7` , xrpl-sdk)
- **Merge pull request #28 from ilubxrp589/m2-requests** (`a0a6f6c` , xrpl-sdk)
- **Merge pull request #27 from ilubxrp589/release-0.2.0** (`c9ccbfe` , xrpl-sdk)
- **release: 0.2.0, the first crates.io release** (`2890e63` , xrpl-sdk)
- **Merge pull request #26 from ilubxrp589/m1-helpers** (`c38fa00` , xrpl-sdk)
- **Merge pull request #25 from ilubxrp589/deps-update** (`004ee04` , xrpl-sdk)
- **Merge pull request #24 from ilubxrp589/sdk-next** (`64541ff` , xrpl-sdk)
- **deps: cargo update; actions/checkout v4 -> v7** (`a9efcd5` , xrpl-sdk)
- **release prep: publish as halcyon-xrpl-*, tel results, drop the stale field table** (`ebe99c6` , xrpl-sdk)
- **docs: CHANGELOG and README for the Client trait, typed objects and amounts** (`b233b21` , xrpl-sdk)
- **docs(readme): current numbers — 34/34 + 261/261 fixtures, 28 builders** (`a2a1464` , xrpl-sdk)
- **chore: enforce the safety rules as workspace lints** (`6e6b6fd` , xrpl-sdk)
- **chore(deps): bump rustls, rustls-webpki, quinn-proto, webpki-roots, rand** (`2853e26` , xrpl-sdk)
- **chore: list the public xrpl.js test seeds in .push-guard-allow** (`4b923d8` , xrpl-sdk)
- **docs: correct TESTING.md's known-address vectors; CHANGELOG Unreleased** (`f1f1722` , xrpl-sdk)
- **chore(core): clear the clippy and rustfmt failures** (`01e823b` , xrpl-sdk)

Independence program

Qualifying days: **0 of 30** (snapshot at 04:21:16 UTC). A UTC day qualifies when every validation is of a ledger our own engine built, with our own transaction order and ledger data taken from peers, and coverage is at least 98%.

UTC day 2026-10-23 — tracked #107,837,738–#107,859,765	Value
Validations	22,028 (22,028 own, 0 borrowed)
Coverage of the tracked ledgers	100.00%

UTC day 2026-10-23 — tracked #107,837,738–#107,859,765	Value
Our own transaction order	on
Ledger data from peers	no

Not qualifying: the ledger data came from the reference node, not peers (Step 4).

The validator saves this record every 64 validations, so a day in progress lags the signing log slightly; it is not cut off at the period's last ledger.

Network

Measure	Value
Ledgers closed	22,045 (#107,841,424–#107,863,468)
Mean close interval	3.92 s
Longest close interval	19 s (before #107,853,159)
Closes at least 15 s apart	1
Empty ledgers	0

- Slow closes 16:43:41 UTC–16:44:00 UTC: ledgers #107,853,159, 1 close at least 15 s apart, up to 19 s.

From ledger close times, which the network records at a 10-second resolution or coarser, so single intervals are approximate.

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_4_0	2026-10-21 14:46 UTC	2026-11-04 14:46 UTC

Amendments enabled on mainnet: 97.

Open items

- Amendment fixCleanup3_4_0: activation due ~2026-11-04 14:46 UTC.

Corrections to earlier reports

None.

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
10-23 05:51	10-24 02:41	xrpl-sdk	`55fb80b`	fix(core): the encoder refuses what rippled refuses instead of changing the transaction
10-23 05:51	10-24 02:41	xrpl-sdk	`37dc627`	fix(core): token values parse and round exactly as rippled reads them
10-23 06:03	10-24 02:41	xrpl-sdk	`1ef2462`	fix(client): submit_and_wait answers only with final outcomes; autofill caps the fee
10-23 06:13	10-24 02:41	xrpl-sdk	`1a10b5a`	fix(crypto): secrets stay out of logs and errors; secp256k1 signs only a 32-byte hash
10-23 06:15	10-24 02:41	xrpl-sdk	`b5fd628`	fix(cli): the server decides whether --yes is needed; failed sends exit non-zero
10-23 06:24	10-24 02:41	xrpl-sdk	`cb9c67c`	fix(client): the WebSocket client closes on drop, notices dead connections, and routes each subscription its own events
10-23 06:29	10-24 02:41	xrpl-sdk	`58116da`	fix(transaction): Batch checks what rippled checks; a transaction that does not parse is Unparsed, not Other
10-23 06:34	10-24 02:41	xrpl-sdk	`eafb7ae`	fix(core): canonical currencies, minimal field headers, capped base58, checked binary amounts
10-23 06:37	10-24 02:41	xrpl-sdk	`6654a7b`	fix(client): paginated calls read one ledger; delivered_amount helpers
10-23 06:40	10-24 02:41	xrpl-sdk	`fd0b1dc`	fix(utils): reserves in exact drops; exact book liquidity; dex floats labeled
10-23 06:43	10-24 02:41	xrpl-sdk	`e288ea4`	fix(core): a ledger entry that does not parse is Unparsed, not Other; from_value is strict
10-23 06:48	10-24 02:41	xrpl-sdk	`c9078f1`	fix(builders): build() checks rippled's rules; AMM mode flags set; AccountTxnID spelled right
10-23 06:52	10-24 02:41	xrpl-sdk	`a7111f2`	feat(core): keylets for every ledger entry type rippled 3.4.1 has
10-23 06:56	10-24 02:41	xrpl-sdk	`99290fc`	chore(lints): deny narrowing `as` casts workspace-wide
10-23 07:09	10-24 02:41	xrpl-sdk	`c0de03c`	docs: 0.6.0: changelog, security policy, CRYPTO.md, parity report; design notes marked historical
10-23 07:09	10-24 02:41	xrpl-sdk	`ababe1b`	fix(codec): read every JSON currency the same way, "1" (noCurrency) included
10-23 07:18	10-24 02:41	xrpl-sdk	`a75ea28`	fix(codec): a token amount may use rippled's badCurrency, as rippled's parser allows
10-23 07:32	10-24 02:41	xrpl-sdk	`52ad1ef`	chore: Cargo.lock for 0.6.0
10-23 12:01	10-24 02:41	xrpl-sdk	`ec20227`	Merge pull request #33 from ilubxrp589/redteam-fixes
10-23 15:50	10-24 02:41	xrpl-sdk	`6e69487`	feat(wallet): open a wallet from a BIP-39 phrase, Xaman secret numbers or an RFC 1751 key
10-23 15:59	10-24 02:41	xrpl-sdk	`73a865a`	fix(client): autofill prices each transaction type as rippled does
10-23 16:01	10-24 02:41	xrpl-sdk	`a8559a9`	fix(client/wasm): wait with the global setTimeout, not window's
10-23 16:19	10-24 02:41	xrpl-sdk	`6cd4393`	feat: sign typed transactions, verify signatures, read balances; BlockingClient complete
10-23 16:24	10-24 02:41	xrpl-sdk	`72e9559`	feat: standard traits on clients and core types
10-23 16:26	10-24 02:41	xrpl-sdk	`5712e4d`	feat(client): Client::pages streams pages with no limit

Committed	Merged into main	Repo	Commit	Subject
10-23 16:30	10-24 02:41	xrpl-sdk	`18f7387`	feat(client): HTTP client builder, polite retries, typed HTTP status
10-23 16:32	10-24 02:41	xrpl-sdk	`5e7e929`	feat(client): XrplWsClient::builder for timeouts, pings and buffers
10-23 16:38	10-24 02:41	xrpl-sdk	`c3ed958`	docs, ci: contributor guide, issue forms, docs.rs labels, CI on three OSes
10-23 16:41	10-24 02:41	xrpl-sdk	`df0e574`	feat(core): ledger entry flags as named constants
10-23 16:48	10-24 02:41	xrpl-sdk	`ad6d7da`	feat: check a ledger against its hash; WS tests that held only on Linux
10-23 17:02	10-24 02:41	xrpl-sdk	`e26fd0f`	docs: an # Errors section on every public function that can fail
10-23 17:18	10-24 02:41	xrpl-sdk	`169824e`	release: 0.7.0; fuzz target for key backups and ledger checks
10-23 17:45	10-24 02:41	xrpl-sdk	`472a78c`	Merge pull request #34 from ilubxrp589/m7-pro
10-23 18:21	10-24 02:41	xrpl-sdk	`7c12fd4`	license: Apache-2.0 with a NOTICE crediting James Turner
10-23 18:24	10-24 02:41	xrpl-sdk	`76a11f1`	Merge pull request #35 from ilubxrp589/apache-license
10-23 18:27	10-24 02:41	xrpl-sdk	`220c123`	feat(client)!: transport errors without reqwest; timeouts are Timeout; caller mistakes are InvalidRequest
10-23 18:33	10-24 02:41	xrpl-sdk	`d2cd7bb`	feat(client)!: Client = every method on any Transport; Send futures; dyn transports
10-23 18:40	10-24 02:41	xrpl-sdk	`2b54449`	feat!: protocol types non_exhaustive, built with new + with_ setters
10-23 18:40	10-24 02:41	xrpl-sdk	`b4bfe6c`	docs(changelog): 0.8 step 3, non_exhaustive types and setters
10-23 18:46	10-24 02:41	xrpl-sdk	`1c14076`	feat!: the typed models are the one transaction API; builders removed
10-23 18:53	10-24 02:41	xrpl-sdk	`761b028`	feat!: typed flags per transaction and ledger entry type
10-23 18:53	10-24 02:41	xrpl-sdk	`dccad87`	chore: versions 0.8.0 (breaking release in progress)
10-23 19:02	10-24 02:41	xrpl-sdk	`701f8e5`	feat!: one Asset type for models, keylets and the client
10-23 19:21	10-24 02:41	xrpl-sdk	`2bd6dea`	feat!: one type per idea, and the remaining API polish
10-23 19:42	10-24 02:41	xrpl-sdk	`af487c8`	feat!: sign with any key: SigningKey and AsyncSigningKey
10-23 20:14	10-24 02:41	xrpl-sdk	`7723826`	feat!: rippled API version 2 by default, and typed metadata
10-23 20:24	10-24 02:41	xrpl-sdk	`b1a3ebf`	feat!: order_book and typed path_find
10-23 20:34	10-24 02:41	xrpl-sdk	`cda0637`	feat!: features that gate code; trimmed dependencies
10-23 21:12	10-24 02:41	xrpl-sdk	`606d285`	fix: review findings before 0.8.0
10-23 21:41	10-24 02:41	xrpl-sdk	`e11b953`	release: 0.8.0

Committed	Merged into main	Repo	Commit	Subject
10-23 21:44	10-24 02:41	xrpl-sdk	`8277ba1`	Merge pull request #36 from ilubxrp589/m8
10-23 22:57	10-24 02:41	xrpl-sdk	`b3f97d9`	docs: benchmarks re-run for 0.8; counts corrected; every CI job listed
10-23 22:57	10-24 02:41	xrpl-sdk	`1f79e48`	ci: a weekly check of rippled's definitions on Devnet, Testnet and mainnet
10-23 22:57	10-24 02:41	xrpl-sdk	`341e037`	perf(core): encode DeliverMax in place; decode addresses with one allocation
10-23 23:05	10-24 02:41	xrpl-sdk	`676d820`	Merge pull request #37 from ilubxrp589/maint
10-23 23:44	10-24 02:41	xrpl-sdk	`0296ebe`	docs(core): an example on every public item in address, codec, crypto, error, maybe_send, flags, wasm_bindgen_exports
10-23 23:45	10-24 02:41	xrpl-sdk	`352e160`	docs(codec): drop a stray doc comment from a private constant
10-23 23:53	10-24 02:41	xrpl-sdk	`68977cc`	docs(client, sdk): an example on every public item in the client's client, blocking, ws, http, stream, faucet and error modules and the SDK
10-23 23:57	10-24 02:41	xrpl-sdk	`d3ec352`	docs(types): an example on every public item in types/, hashes.rs, ledger.rs, transaction/{batch,cost,metadata,mod}.rs and utils/
10-23 23:57	10-24 02:41	xrpl-sdk	`0cb6b60`	fix(client): ManifestEvent keeps the serialized manifest
10-24 00:00	10-24 02:41	xrpl-sdk	`dd60034`	fix(codec): TransactionResult by name, as rippled writes and reads it
10-24 00:00	10-24 02:41	xrpl-sdk	`8a7d0e4`	docs(client): an example on every public item in requests.rs and types/
10-24 00:01	10-24 02:41	xrpl-sdk	`fe1ee9b`	docs(client): six response docs that disagreed with rippled's answers
10-24 00:01	10-24 02:41	xrpl-sdk	`84ba2e8`	docs(models): an example on every public item in transaction/models/ and ledger_object/formats.rs
10-24 00:01	10-24 02:41	xrpl-sdk	`0f70bdd`	chore: stop tracking Python bytecode (**pycache**)
10-24 00:57	10-24 02:41	xrpl-sdk	`0d056ba`	fix(models): Batch::validate runs rippled's ledger-free Batch checks
10-24 00:59	10-24 02:41	xrpl-sdk	`8144d21`	style: rustfmt
10-24 01:03	10-24 02:41	xrpl-sdk	`73572fb`	fix(core): no unused import in a no_std build
10-24 01:03	10-24 02:41	xrpl-sdk	`79859c5`	ci: warnings fail the MSRV / no_std job
10-24 01:04	10-24 02:41	xrpl-sdk	`8b79f6c`	docs: changelog and roadmap for the examples pass
10-24 01:15	10-24 02:41	xrpl-sdk	`3c06631`	Merge pull request #38 from ilubxrp589/docs-examples
10-24 02:08	10-24 02:41	xrpl-sdk	`a205e5e`	release: 0.8.1
10-24 02:17	10-24 02:41	xrpl-sdk	`9279f39`	Merge pull request #39 from ilubxrp589/release-0.8.1
10-22 22:00	10-24 02:41	xrpl-sdk	`4fdc498`	Merge pull request #32 from ilubxrp589/m5-trust
10-22 21:35	10-24 02:41	xrpl-sdk	`86cadcb`	feat: 0.5.0, trust: fuzzing, benchmarks, MSRV, rippled parity at the edges

Committed	Merged into main	Repo	Commit	Subject
10-22 19:29	10-24 02:41	xrpl-sdk	`5f587bf`	Merge pull request #31 from ilubxrp589/m4-dx
10-22 18:53	10-24 02:41	xrpl-sdk	`ef5e034`	feat: cookbook, Amount helpers, every public item documented; 0.4.0
10-22 18:24	10-24 02:41	xrpl-sdk	`dac53ff`	feat: hxrpl, the command line (new crate halcyon-xrpl-cli)
10-22 18:18	10-24 02:41	xrpl-sdk	`3864bfa`	feat(client): BlockingClient — every Client method as a plain call
10-22 18:15	10-24 02:41	xrpl-sdk	`a68a422`	feat(client): typed WebSocket stream events for every rippled stream
10-22 18:05	10-24 02:41	xrpl-sdk	`eb2c287`	Merge pull request #30 from ilubxrp589/m3-transactions
10-22 18:04	10-24 02:41	xrpl-sdk	`3f39e24`	feat: typed models for all 82 transaction types, and Batch (XLS-56); 0.3.0
10-22 17:41	10-24 02:41	xrpl-sdk	`07eab00`	Merge pull request #29 from ilubxrp589/post-release-0.2.0
10-22 17:37	10-24 02:41	xrpl-sdk	`d98c5d7`	docs: 0.2.0 is out — CHANGELOG, ROADMAP, README badges
10-22 17:31	10-24 02:41	xrpl-sdk	`a0a6f6c`	Merge pull request #28 from ilubxrp589/m2-requests
10-22 17:30	10-24 02:41	xrpl-sdk	`1aed483`	feat(client): a typed request and response for every public rippled method
10-22 17:09	10-24 02:41	xrpl-sdk	`c9ccbfe`	Merge pull request #27 from ilubxrp589/release-0.2.0
10-22 17:06	10-24 02:41	xrpl-sdk	`2890e63`	release: 0.2.0, the first crates.io release
10-22 17:05	10-24 02:41	xrpl-sdk	`c38fa00`	Merge pull request #26 from ilubxrp589/m1-helpers
10-22 17:03	10-24 02:41	xrpl-sdk	`79f4949`	feat: faucet, Wallet::seed, live Testnet test; fix fee and UInt64; full docs
10-22 16:44	10-24 02:41	xrpl-sdk	`42fe58a`	feat(core): transaction hashes, ledger entry IDs, channel claims, everyday utils
10-22 16:26	10-24 02:41	xrpl-sdk	`004ee04`	Merge pull request #25 from ilubxrp589/deps-update
10-22 16:03	10-24 02:41	xrpl-sdk	`64541ff`	Merge pull request #24 from ilubxrp589/sdk-next
10-22 16:01	10-24 02:41	xrpl-sdk	`a9efcd5`	deps: cargo update; actions/checkout v4 -> v7
10-22 15:39	10-24 02:41	xrpl-sdk	`ebe99c6`	release prep: publish as halcyon-xrpl-*, tel results, drop the stale field table
10-22 12:19	10-24 02:41	xrpl-sdk	`b233b21`	docs: CHANGELOG and README for the Client trait, typed objects and amounts
10-22 12:18	10-24 02:41	xrpl-sdk	`9bf6d7b`	feat: typed ledger objects and typed amounts (MPT included)
10-22 12:11	10-24 02:41	xrpl-sdk	`5073f6e`	feat(client): one Client trait over HTTP, WS and wasm WS; typed requests
10-22 12:07	10-24 02:41	xrpl-sdk	`9252046`	fix(wasm): the wasm build compiles again; CI checks it
10-22 12:03	10-24 02:41	xrpl-sdk	`a2a1464`	docs(readme): current numbers — 34/34 + 261/261 fixtures, 28 builders

Committed	Merged into main	Repo	Commit	Subject
10-22 12:03	10-24 02:41	xrpl-sdk	`6e6b6fd`	chore: enforce the safety rules as workspace lints
10-22 11:58	10-24 02:41	xrpl-sdk	`eaece20`	feat(codec): rippled 3.4.1 parity — Int32, PermissionValue names, X-addresses
10-22 11:46	10-24 02:41	xrpl-sdk	`2853e26`	chore(deps): bump rustls, rustls-webpki, quinn-proto, webpki-roots, rand
10-22 11:39	10-24 02:41	xrpl-sdk	`4b923d8`	chore: list the public xrpl.js test seeds in .push-guard-allow
10-22 11:38	10-24 02:41	xrpl-sdk	`f1f1722`	docs: correct TESTING.md's known-address vectors; CHANGELOG Unreleased
10-22 11:38	10-24 02:41	xrpl-sdk	`bdb2a59`	fix(core): Ed25519 signs the signing data itself, as rippled verifies it
10-22 11:19	10-24 02:41	xrpl-sdk	`01e823b`	chore(core): clear the clippy and rustfmt failures
10-22 11:19	10-24 02:41	xrpl-sdk	`24f3af2`	fix(sdk): order multisign Signers by AccountID bytes, not base58 text

Times in UTC. A commit appears on the day it was committed to main or merged into it.